

Entregable — Frontend (Ingeniero de Software, UI)

Integrante: Keyner Trujillo **Rol:** Desarrollador Frontend · apoyo UX/UI **Proyecto:** Muelle Digital

1. Tecnologías utilizadas

Tecnología	Para qué
React 18	Librería de UI basada en componentes
Vite 5	Bundler/dev server ultrarrápido
TailwindCSS 3	Estilos utilitarios + tema de marca + modo oscuro
React Router 6	Navegación entre páginas (SPA)
Axios	Cliente HTTP para consumir la API REST
MapLibre GL JS	Mapas interactivos (punto de encuentro)
lucide-react	Iconografía

2. Arquitectura del frontend

```
frontend/src/ |— main.jsx           ← punto de entrada
(Router + AuthProvider) |— App.jsx   ← definición de
rutas |— index.css                 ← Tailwind + clases de marca
(.btn, .card, .input) |— api/client.js ← instancia Axios
con token JWT automático |— context/AuthContext.jsx ← estado
global de sesión |— components/    ← reutilizables
(Navbar, Logo, MapaPunto, etc.) |— pages/
vistas (Landing, Login, Home, Detalle, Paneles...) ←
```

Patrones de diseño aplicados:

- **Componentes reutilizables:** Logo, RatingStars, MapaPunto, SelectorPunto, ImageWithFallback, FeaturesGrid, ExperienceDetailPanel.
- **Context API** para autenticación global (evita prop drilling).
- **Rutas protegidas** (ProtectedRoute) por sesión y por rol.

- **Componente de presentación compartido** (ExperienceDetailPanel) reutilizado en el catálogo y en la página de detalle (DRY).
- **Interceptores Axios:** adjuntan el token y normalizan errores en un solo lugar.

3. Funcionalidades implementadas

- **Landing** con hero, grid responsivo de experiencias y CTA.
- **Login / Registro** con diseño de pantalla partida (estilo moderno).
- **Catálogo maestro-detalle:** lista a la izquierda, detalle a la derecha con mapa, reserva y reseñas.
- **Reserva + pago** (redirección a Stripe Checkout y métodos simulados).
- **Reseñas en tiempo real** (refresco automático por polling).
- **Mapas:** el guía marca el punto de encuentro; el turista lo visualiza.
- **Paneles de guía y administrador** con pestañas.
- **Modo claro/oscuro** (tema morado de marca, persistente).

4. Integración con el backend

- Toda la comunicación es vía **API REST** con Axios (VITE_API_URL).
- El token JWT se guarda en localStorage y se adjunta automáticamente en cada petición (interceptor).
- Ejemplo de consumo:

```
const { data } = await api.get('/experiencias', { params: filtros
});
```

5. Manejo de errores y casos borde

- **Interceptor de respuesta** convierte cualquier error de la API en un mensaje legible.
- Estados de **carga** ("Cargando...") y **vacío** ("No se encontraron experiencias").
- **Validaciones de formulario:** contraseña mínima, campos obligatorios.
- **ImageWithFallback:** si una imagen no carga, muestra un degradado de marca.
- **Rutas protegidas:** redirige a login si no hay sesión, o a inicio si el rol no tiene permiso.

6. UX/UI — Sistema de diseño

- **Identidad de marca:** logo oficial logomuelle.png con contorno blanco para visibilidad sobre fondos oscuros.
- **Paleta morada** de marca (definida en tailwind.config.js) con modo claro y oscuro.
- **Clases base** reutilizables en index.css: .btn-primary, .btn-outline, .card, .input, .label.

- **Responsive:** grid de 1 → 2 → 4 columnas según el dispositivo; catálogo adaptable.
- **Accesibilidad básica:** etiquetas en formularios, contraste, aria-label en controles.

7. Guía de instalación (frontend)

```
cd frontend npm install cp .env.example .env #  
VITE_API_URL=http://localhost:4000/api npm run dev  
# http://localhost:5173
```

Build de producción: npm run build → carpeta dist/.

8. Herramientas a demostrar

- **IDE:** Visual Studio Code.
- **Control de versiones:** Git + GitHub (ramas feature/*).
- **Navegador + DevTools** para depurar componentes y red.
- **Vite** (HMR) para desarrollo en caliente.



Guion de presentación (3-5 min)

1. **(30s)** "Soy Keyner, desarrollador frontend; construí la interfaz en React."
2. **(2 min)** Demo en vivo: navego el catálogo, abro una experiencia (muestro el **mapa** y las **reseñas en vivo**), alterno **modo claro/oscuro**.
3. **(1 min)** Cómo consumo la API del backend de Esteban y cómo reutilizo componentes.
4. **(30s)** Preguntas: manejo de estado, rutas protegidas, responsive.